# 3.5 Các Hàm Kích Hoạt Phổ Biến

- **Sigmoid:** Ép đầu ra về khoảng [0, 1]. Dùng cho phân loại nhị phân.
- **Tanh:** Ép về [-1, 1], có tâm tại 0.
- **ReLU:** Bằng 0 nếu âm, giữ nguyên nếu dương. Nhanh, đạo hàm tốt, là mặc định cho các lớp ẩn hiện nay.

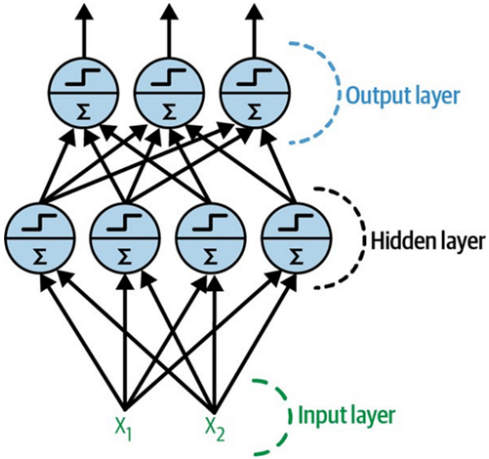



Figure: Hình 10-7: Bảng các Hàm kích hoạt (Activation Functions)

# 4.1 MLP Hồi quy (Regression MLP)

- Hồi quy dự đoán một giá trị liên tục (Ví dụ: Giá nhà).
- Lớp đầu ra chỉ cần **1 nơ-ron duy nhất** (Không dùng hàm kích hoạt).
- Nếu dự đoán giá trị phải luôn dương, dùng hàm kích hoạt **ReLU** hoặc **Softplus** ở đầu ra.
- Hàm mất mát (Loss function): **MSE (Mean Squared Error)** hoặc **MAE (Mean Absolute Error)**.

# 4.2 Cấu hình chuẩn cho MLP Hồi quy

| **Hyperparameter** | **Typical Value** |
|---|---|
| Nơ-ron đầu vào (Input) | Một nơ-ron cho mỗi đặc trưng (Feature). |
| Số lớp ẩn (Hidden Layers) | Tùy thuộc bài toán, thường từ 1 đến 5. |
| Nơ-ron mỗi lớp ẩn | Thường từ 10 đến 100. |
| Nơ-ron đầu ra (Output) | 1 (Nếu tiên đoán 1 giá trị), N (Nếu tiên đoán N chiều). |
| Hàm kích hoạt (Lớp ẩn) | ReLU (Hoặc SELU, Mish). |
| Hàm kích hoạt (Đầu ra) | None (Liên tục), ReLU (Dương), Sigmoid (Khoảng 0-1). |
| Hàm mất mát (Loss) | MSE, MAE, hoặc Huber Loss. |

Table: Hình 10-8: Kiến trúc MLP Hồi quy chuẩn

# 4.3 MLP Phân loại (Classification MLP)

- **Phân loại nhị phân (Binary):** Dùng 1 nơ-ron đầu ra với hàm **Sigmoid**. (Xác suất từ 0 đến 1). Loss: Binary Crossentropy.
- **Phân loại đa nhãn (Multilabel):** Vd nhận dạng email vừa là spam vừa là urgent. Dùng N nơ-ron đầu ra, mỗi nơ-ron có hàm **Sigmoid**. Loss: Binary Crossentropy.
- **Phân loại đa lớp (Multiclass):** Vd nhận diện MNIST (Từ 0 đến 9). Dùng N nơ-ron đầu ra với hàm **Softmax**. Loss: Categorical Crossentropy.

# 4.4 Cấu hình chuẩn cho MLP Phân loại

| **Hyperparameter** | **Binary** | **Multilabel** | **Multiclass** |
|---|---|---|---|
| Input & Hidden | Giống như Hồi quy (1-5 lớp, ReLU) | | |
| Output Neurons | 1 | 1 cho mỗi nhãn | 1 cho mỗi lớp (Tổng cộng N) |
| Output Activation | **Sigmoid** | **Sigmoid** | **Softmax** |
| Loss Function | Binary Crossentropy | Binary Crossentropy | Categorical Crossentropy |

Table: Hình 10-9: Kiến trúc MLP Phân loại chuẩn

# 5.1 TensorFlow và Keras

- **TensorFlow:** Thư viện mã nguồn mở cho Học sâu của Google. Xử lý đồ thị tính toán mạnh mẽ trên CPU, GPU, TPU.
- **Keras:** API bậc cao (High-level API) của TensorFlow. Cực kỳ thân thiện với người dùng, dễ học, nhưng đủ linh hoạt để tuỳ chỉnh mô hình chuyên sâu.
- Hướng dẫn sau sử dụng bộ dữ liệu **Fashion MNIST** (Nhận diện ảnh 28x28 của các mặt hàng quần áo thời trang).

# 5.2 Sequential API: Xây dựng mô hình

- Keras `Sequential` API tạo mô hình bằng cách xếp chồng các lớp (Layer) tuần tự.

```
import tensorflow as tf
from tensorflow import keras

model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu"),
    keras.layers.Dense(100, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
])
model.summary() # Xem tom tat cau truc mo hinh
```

# 5.3 Giải thích Mô hình Sequential

- `Flatten`: Lớp chuyển đổi ảnh ma trận 2D (28x28) thành mảng 1D (784). (Không có trọng số).
- `Dense(300)`: Lớp ẩn đầu tiên với 300 nơ-ron kết nối đầy đủ. Số tham số = 784 x 300 + 300 (bias) = 235,500.
- `Dense(100)`: Lớp ẩn thứ hai với 100 nơ-ron. (30,100 tham số).
- `Dense(10)`: Lớp đầu ra với 10 nơ-ron (cho 10 lớp mặt hàng). Dùng Softmax để xuất ra xác suất tổng bằng 1.

# 5.4 Biên dịch (Compile) và Huấn luyện (Fit)

```
# Buoc 1: Compile - Khai bao Loss va Optimizer
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="sgd",
              metrics=["accuracy"])

# Buoc 2: Fit - Huan luyen mo hinh (Lan truyen nguooc)
history = model.fit(X_train, y_train, epochs=30,
                    validation_data=(X_valid, y_valid))
```

- `sparse_categorical_crossentropy`: Dùng vì nhãn Y là các số nguyên rải rác (0-9).
- `sgd`: Stochastic Gradient Descent.
- `epochs=30`: Huấn luyện duyệt qua toàn bộ tập dữ liệu 30 lần.

# 5.5 Đường cong Học tập (Learning Curves)

- Object `history` chứa dữ liệu loss và accuracy của train/valid sau mỗi epoch.
- Nhìn vào biểu đồ, Loss liên tục giảm dần (tốt).
- Accuracy liên tục tăng (tốt).
- Nếu Validation Loss đi ngang hoặc tăng lên, mô hình đang bị **Overfitting**.

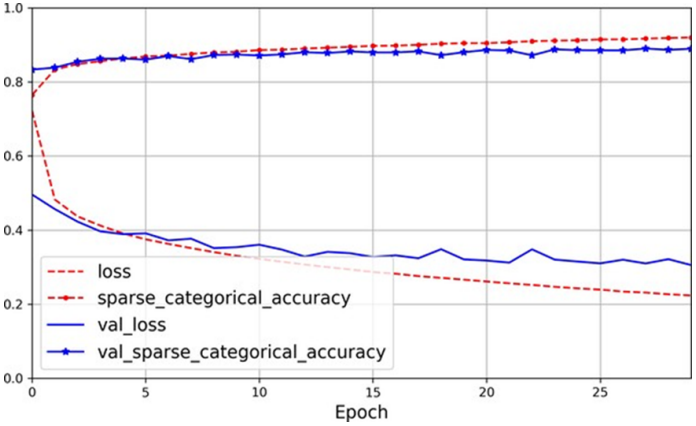



Figure: Hình 10-11: Đồ thị Loss và Accuracy theo số vòng lặp (Epoch)

# 6.1 Vượt giới hạn với Functional API

- Mô hình tuần tự (Sequential) rất phổ biến, nhưng đôi khi ta cần cấu trúc phi tuần tự.
- Ví dụ: **Wide & Deep Neural Network**.
- Kiến trúc này nối trực tiếp lớp Đầu vào (Input) với lớp Đầu ra (Output) song song cùng với Lớp ẩn sâu.
- Cho phép mạng học được cả **Mẫu sâu (Deep patterns)** lẫn **Quy luật nông (Simple rules)**.

# 6.2 Kiến trúc Wide & Deep

- Các mô hình MLP thông thường ép mọi dữ liệu chảy qua một chuỗi các lớp ẩn, khiến các quy luật đơn giản ban đầu có thể bị biến dạng hoặc "quên" mất.
- Kiến trúc Wide & Deep chia nhánh để bảo toàn các đặc trưng đầu vào gốc.

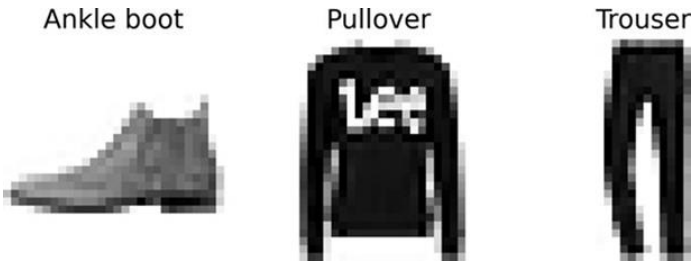



Figure: Hình 10-12: Mạng Nơ-ron Rộng và Sâu (Wide & Deep)

# 6.3 Code: Functional API

```
input_ = keras.layers.Input(shape=X_train.shape[1:])
hidden1 = keras.layers.Dense(30, activation="relu")(input_)
hidden2 = keras.layers.Dense(30, activation="relu")(hidden1)

# Ghep noi (Concatenate) duong "Wide" va "Deep"
concat = keras.layers.Concatenate()([input_, hidden2])

output = keras.layers.Dense(1)(concat)
model = keras.models.Model(inputs=[input_], outputs=[output])
```

- Trong Functional API, ta phải gọi từng lớp như một hàm và truyền input vào.
- Cuối cùng, bọc tất cả lại trong `keras.models.Model`.

# 6.4 Functional API: Nhiều luồng dữ liệu (Multiple Inputs)

- Sẽ ra sao nếu ta muốn gửi một nhóm đặc trưng vào "Đường rộng", và một nhóm đặc trưng khác vào "Đường sâu"?
- Keras Functional API có thể tạo nhiều luồng Input.
- Lúc gọi hàm `fit()`, ta sẽ truyền `(X_train_A, X_train_B)`.

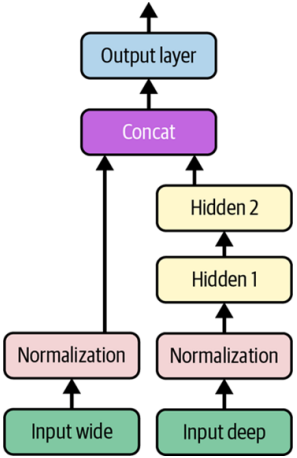



Figure: Hình 10-14: Xử lý nhiều luồng đầu vào

# 6.5 Functional API: Nhiều đầu ra (Multiple Outputs)

- Tương tự, một mạng cũng có thể có nhiều đầu ra.
- Ví dụ: Phát hiện vật thể trên ảnh (1 Output là Phân loại vật, 1 Output khác là Tọa độ đóng khung vật).
- Mỗi Output sẽ có một `loss` riêng biệt. Keras tính toán theo tổng các loss có hệ số nhân.

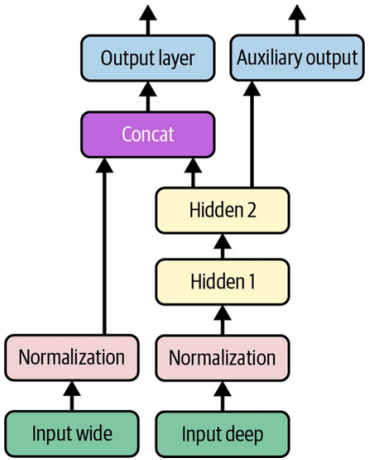



Figure: Hình 10-15: Mạng có nhiều đầu ra